RESEARCH MEETING

Heterogeneous Model Serving at Different Granularities: Helix and Tessera

Helix (ASPLOS 2025) + Tessera

Yu-Jui Liang
Advisor: Hsiang-Yun Cheng
National Taiwan University | Academia Sinica
27 Oct 2026

Helix: Yixuan Mei et al., ASPLOS 2025

Tessera: Tiancheng Hu et al., arXiv:2604.10180v2

Put the Two Papers on One Map

	Helix	Tessera
Heterogeneity	GPU capability + network topology	Kernel-specific GPU preference
Optimization unit	Layer placement + request pipeline	GPU kernel
Main bottleneck	Compute + network capacity	Compute/memory characteristics + communication
Offline planning	MILP placement	Analyzer + DDG + MILP
Runtime	Request path scheduling	Stored policy selection / switching

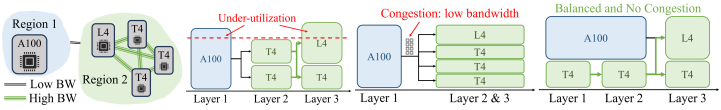
Helix asks where layers should be placed and how requests should flow. Tessera asks which GPU should execute each kernel.

These are different optimization dimensions, not a coarse-to-fine ranking.

Source: Our synthesis. Helix Sec. 4–5; Tessera Sec. III.

Helix: What Problem Is It Solving?

Helix jointly optimizes model-layer placement and request routing for heterogeneous, potentially geo-distributed GPU clusters.



(a) Five nodes in two clusters. **(b)** Uniform model partition. **(c)** Balanced FLOPs. **(d)** Network aware placement.

Figure 1. Examples of sub-optimal model placement and request schedule. 1a) all GPUs and network condition in this example. The order of compute capacity is: A100 > L4 > T4; 1b) Model placement by uniformly partition the model, then allocate devices by a balanced compute capacity; 1c) Co-optimizing model partition and device placement to make the compute capacity more balanced; 1d) Co-optimizing model partition, device placement, and request scheduling in a network-aware way.

Because

- GPU types, compute, and memory differ.
- Clusters may also sit in different regions, so link bandwidth differs. That is geo-distribution, not another word for heterogeneity.

Therefore

- A FLOP-proportional cut only balances compute.
- If every activation still crosses a slow link, the network becomes the bottleneck.

A balanced compute partition is not a maximum-throughput serving plan.

Source: Helix Sec. 1, 4.2; Fig. 1. Panel (c) balances FLOPs and still congests the slow link.

Helix: How Does It Work?

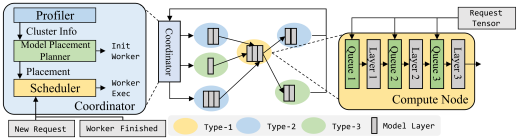


Figure 3. Helix overview. In Helix, the coordinator plans model placement as described in Sec. 4.4. We only need to run model placement once for each cluster. When a new request arrives, the coordinator node runs Helix scheduler to assign it a per-request pipeline and sends it to the first node in the pipeline. Each compute node in the pipeline performs inference on the request on the layers it is responsible for and sends the (output for the) request to the next node in the pipeline. When the last node in the pipeline finishes performing inference on its layers, it will send the output token for the request to the coordinator (Worker Finished). The coordinator schedules generation of the next token for the request using the same pipeline.

Edge capacity = tokens/s through one compute or network resource. For a fixed placement, source–sink max flow estimates sustainable throughput.

Offline planning

1. Profile GPU compute, memory, and links.
2. MILP chooses the layer placement.
3. Represent that placement as a flow graph.
4. Max-flow gives token rates and path budgets.

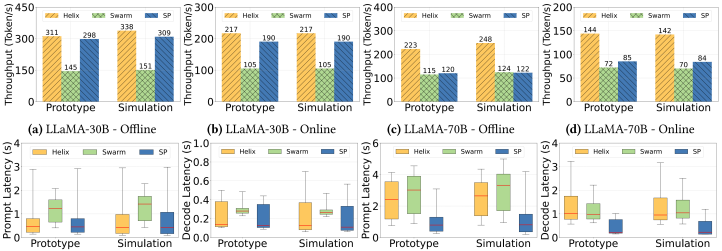
Online runtime

5. Assign each request one pipeline.
6. Later decode tokens reuse that pipeline.

Max-flow is a throughput model, not a latency model. The MILP is not rerun per request.

Source: Helix Sec. 4.3–5, Fig. 3. Offline/online here mean when the optimization runs.

Q1: Does Helix Improve End-to-End Serving on Real Hardware?



(a) LLaMA-30B - Offline (b) LLaMA-30B - Online (c) LLaMA-70B - Offline (d) LLaMA-70B - Online

(e) LLaMA-30B - Prompt (f) LLaMA-30B - Decode (g) LLaMA-70B - Prompt (h) LLaMA-70B - Decode

Figure 6. Single cluster results: (a - d) decode throughput, **(e - h)** prompt and decode latency for online serving. The box shows the 25 and 75 percentile. The whisker shows the 5 and 95 percentile. The red line shows median.

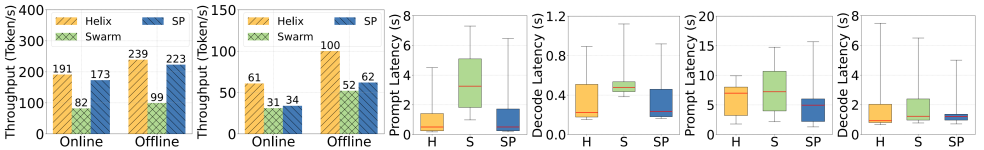
Prototype, one region, 10 Gb/s: 4 A100 + 8 L4 + 12 T4. LLaMA 30B/70B vs. SP and Swarm. **Offline workload:** requests stay available. Read saturated decode throughput. **Online workload:** Azure arrivals over time. Throughput, queueing, and latency all matter. Vs. Swarm, decode throughput is about 2×. Latency is not always lower.

The prototype gains throughput. The objective does not guarantee a latency win.

Source: Helix Sec. 6.2–6.3, Fig. 6. Simulation columns check the simulator. Workload ≠ planner timing.

Q2: Does Network-Aware Placement Matter under Slow Inter-Cluster Links?

Question: If the inter-cluster network is slow, does putting it inside placement actually matter?



(a) LLaMA-30B (b) LLaMA-70B (c) LLaMA-30B (d) LLaMA-30B (e) LLaMA-70B (f) LLaMA-70B

Figure 7. Geo-distributed clusters: (a - b) decode throughput, **(c - f)** prompt and decode latency for online serving. "H" stands for Helix, "S" stands for Swarm. Box: 25 and 75 percentile. Whisker: 5 and 95 percentile. Red line: median.

- **Simulator, not prototype.** Same GPUs, three clusters, about 100 Mb/s and 50 ms between clusters.
- LLaMA 30B vs. Swarm: about 2.3–2.4× decode throughput in both offline and online workloads.
- Those 30B **online** latency reductions vs. Swarm are 66% prompt and 24% decode.

When the network is the bottleneck, joint placement and routing matter much more.

Source: Helix Sec. 6.2, 6.4, Fig. 7. Do not read these latency drops as prototype results.

Q3: Where Do the Gains Come From: Placement or Scheduling?

Question: Are the gains from placement, scheduling, or both? Offline LLaMA 70B. Metric: decode tokens/s.

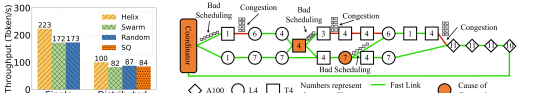
Placement ablation. Helix scheduler fixed.
vs. Petals / Swarm
Single: 1.23× / 2.10× Geo: 1.49× / 2.38×



(a) Decode throughput. **(b)** Model placement case study. Numbers on each node represent the number of layers the node holds.

Figure 9. (a) Comparing different model placement methods with offline serving of LLaMA 70B. **(b)** The case study shows model placements for serving LLaMA 70B on the single cluster setup.

Scheduling ablation. Helix placement fixed.
vs. Swarm / random
Single: +30% / +29% Geo: +22% / +15%



(a) Decode throughput. **(b)** Request scheduling case study.

Figure 10. (a) Comparing different request scheduling methods with offline serving of LLaMA 70B. **(b)** A case study that illustrates the congestion in Swarm and random scheduling when serving LLaMA 70B in geo-distributed clusters setup.

Placement builds the resource graph. Scheduling is what actually uses that capacity.

Source: Helix Sec. 6.6–6.7, Figs. 9–10. Geo-distributed bars are simulator results.

Tessera: Why Is Phase-Level Disaggregation Too Coarse?

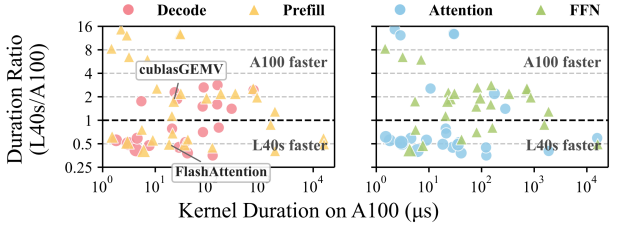


Fig. 3: Kernel duration ratio (L40s / A100) for GPT-oss 20B inference, grouped by (a) prefill and decode phases and (b) attention and FFN blocks. Kernels above the dashed line run faster on A100, while those below run faster on L40s.

GPT-oss 20B, A100 vs. L40s. About **45% of prefill** kernels and **57% of decode** kernels are faster on L40s. Attention and FFN blocks are mixed too: a block name does not pick the GPU. Points below the dashed line prefer L40s. The 45%/57% figures count kernels, not execution time.

A phase label is not a hardware preference.

Source: Tessera Sec. II-C, Fig. 3. X-axis: kernel duration on A100. Y-axis: L40s/A100 time.

Tessera: How Does It Work?

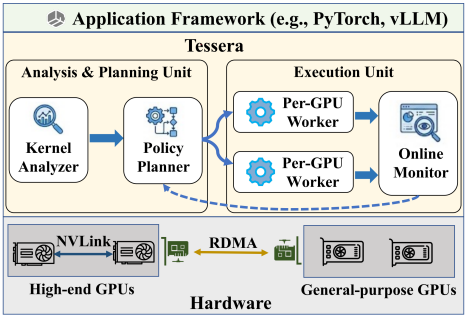


Fig. 4: Design overview of Tessera.

1. **Analyzer:** memory ranges and RAW dependencies. If B reads what A wrote, B waits for that data.
2. **Profiling:** each kernel's latency on each GPU type.
3. **Planner:** build DDGs and solve offline policies.
4. **Runtime:** workers run assigned kernels and transfers. The monitor selects a stored policy.

Throughput policy: balance compute and communication when requests can overlap. **Latency policy:** at low load, shorten one request's compute plus the transfers that cannot be hidden.

Analysis and MILP stay offline. Runtime only executes or switches stored policies.

Source: Tessera Sec. III, Fig. 4. RAW = Read After Write. The latency objective sums kernel and cut-edge costs.

Q1: Does Kernel-Level Disaggregation Outperform Phase/Block-Level Methods?

Question: Does scheduling at kernel granularity raise end-to-end throughput, or only individual kernel speed?

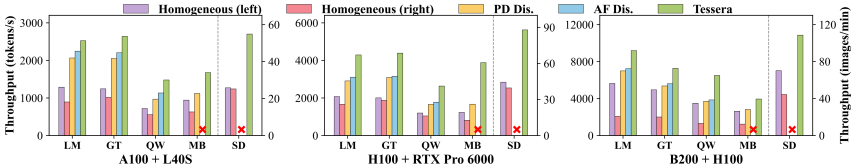


Fig. 6: Offline throughput across five workloads and three heterogeneous GPU pairs. Left y-axis: tokens/s for LLM, MLLM, and SSM workloads, right y-axis: images/min for SD. Red X marks indicate that the disaggregation method is inapplicable.

Five offline workloads, three GPU pairs. PD = prefill/decode. AF = attention/FFN.

- Vs. PD: 1.5× average, up to 2.3× throughput.
- Vs. AF: 1.35× average, up to 1.7× throughput.

Kernel-level scheduling turns the motivation into end-to-end throughput gains.

Source: Tessera Sec. V-A–B, Fig. 6. Tokens/s; SD axis is images/min. Red X: baseline inapplicable.

Q2: Can Tessera Sustain More Requests under the Same Latency SLO?

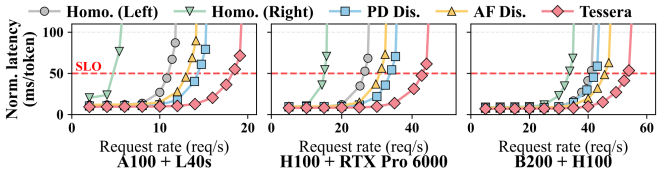


Fig. 7: Online normalized latency on GPT-oss 20B across three GPU pairs. The red dashed line marks the SLO target (50 ms/token).

GPT-oss 20B, Poisson arrivals, three GPU pairs. **Low load:** normalized latency about 1.3× / 1.2× lower than PD / AF. **Same 50 ms/token SLO:** up to about 1.3× the request rate of the best baseline. Offline throughput and this SLO capacity are different regimes.

The gain is not only saturated throughput. Stored policies also help online.

Source: Tessera Sec. V-B, Fig. 7. Normalized latency = end-to-end time / output tokens.

Q3: Does Communication Overhead Cancel the Benefit of Kernel Splitting?

Question: Kernel splits add cross-GPU transfers. Can pipelining hide enough of that cost?

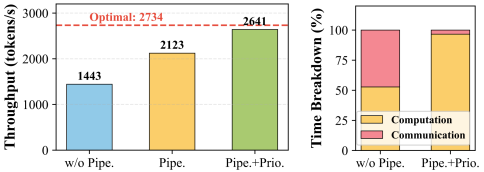


Fig. 9: (a) Throughput under different pipeline configurations. (b) Time breakdown on the bottleneck GPU.

GPT-oss 20B, offline. No pipeline 1443; pipeline 2123; pipeline + priority **2641 tokens/s**.

- Zero-communication reference: 2734 tokens/s. $2641/2734 \approx 96.6\%$.
- Later requests use a lower CUDA stream priority, so their transfers do not all line up.

Placement alone is not enough. Overlap is a necessary runtime mechanism.

Source: Tessera Sec. III-C, V-D, Fig. 9. The reference assumes zero communication cost. GPU pair not named here.

What Do Helix and Tessera Teach Us about Heterogeneous Serving?

System	Optimization unit	Main question
ThunderServe	Prefill / decode phase	Which GPU group should serve each phase?
Helix	Layers + request pipeline	Where should layers go, and how should requests flow?
Hetis	Attention heads	How should attention work and KV state be distributed?
Tessera	GPU kernels	Which GPU should execute each kernel?

Conclusion: These systems optimize heterogeneity along different dimensions.

Finer granularity can expose hardware opportunities, but adds dependency tracking, communication, planning, and runtime coordination.

How fine should optimization become before communication and coordination costs outweigh the benefit?

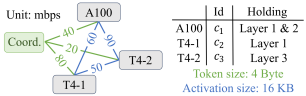
Source: Our project-level synthesis and open question, not a cross-paper performance ranking.

Max-Flow Graph Abstraction

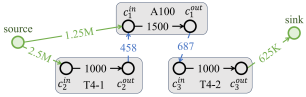
HELIX / FORMULATION

- Each compute node becomes $c^{\text{in}} \rightarrow c^{\text{out}}$.
- The coordinator becomes the source and the sink.
- An edge capacity is how many tokens may cross it per second.
- Max flow from source to sink is the serving throughput of this placement.

The graph is an accounting device for one fixed placement.



(a) A 3-node cluster with model placement. Network connections between the coordinator and compute nodes transmit tokens (4 Byte) while others transmit intermediate activations (16 KB)



(b) Graph abstraction of the cluster.

Figure 2. Graph abstraction of a 3-node cluster with given model placement. Numbers on the edges in Fig. 2b represent their capacity, which is the number of tokens that can pass through the edges per second. Max flow between source and sink equals the max serving throughput of the cluster.

Fig. 2. Capacities are tokens/s.

Source: Sec. 4.3, Fig. 2. PDF p. 4.

Modeling Compute Capacity

HELIX / FORMULATION

NODE EDGE

$c^{\text{in}} \rightarrow c^{\text{out}}$

Capacity is the minimum of profiled compute throughput and the node's network throughput.

LAYER COUNT

T_j tokens/s

T_j is the profiled rate when that node holds j layers. It becomes the MILP compute limit.

PROFILING

Once per node

Helix measures throughput rather than using the datasheet TFLOP/s from Table 3.

NOT A GPU

Source and sink

The coordinator injects tokens and collects tokens. It is not given a compute edge.

Compute capacity is a profiled token rate, and it already takes the node's network limit into account.

Source: Sec. 4.3. T_j is defined with Constraint 3 in Sec. 4.4.

Modeling Network Capacity

HELIX / FORMULATION

A link is valid only in one of three cases:

1. Coordinator $\rightarrow$ a node that holds the first layer. Payload is a token (Fig. 2 uses 4 bytes).
2. A node that holds the last layer $\rightarrow$ coordinator. Payload is again a token.
3. Node $i \rightarrow$ node j when j holds the layers needed immediately after i. Payload is an activation (Fig. 2 uses 16 KB).

$$\text{capacity (tokens/s)} = \frac{\text{profiled average bandwidth}}{\text{bytes per token or activation}}.$$

The formula is the paper's rule. The byte sizes above are the Fig. 2 annotations, not universal constants.

Activations cost far more tokens of budget than the final output token, so wide-area links saturate first.

Source: Sec. 4.3. Bandwidth is a one-time average, not a runtime measurement.

Why Max Flow Represents Serving Throughput

HELIX / FORMULATION

- One unit of flow is one token that can pass a node or a link in one second.
- Conservation forces tokens into c^{in} to leave through c^{out}.
- Helix computes the source–sink max flow with preflow-push. That value is the placement's max serving throughput.
- The model is a steady token rate. It does not output prompt latency, decode latency, or a percentile.
- A later early-stop bound is the sum of node compute rates, divided by the number of layers.

Max flow answers "how many tokens/s can this placement carry?" It is not a latency model.

Source: Sec. 4.3 and the early-stop remark in Sec. 4.5.

MILP Variables and Objective

HELIX / MILP

Symbol	Role	Meaning
s_i	integer	first layer on node i
b_{ij}	binary	node i holds exactly j layers
$f_{i,j}$	real	flow from i to j, in tokens/s
$d_{i,j}$	binary	whether link (i, j) is valid
cond	binary	auxiliaries that linearize overlap

$e_i = s_i + \sum_j j\, b_{ij}$. Node i holds the half-open range $[s_i, e_i)$. k is the most layers that fit on that GPU.

$$\max \sum_i f_{\text{source},i}$$

The objective maximizes coordinator outflow. Table 5 groups the variable counts as linear in $|C|$ or $|E|$.

Source: Sec. 4.4, Tables 5–6. Notation here is compacted from the paper.

Placement and Memory Constraints

HELIX / MILP

$$\sum_{j=1}^{k} b_{ij} = 1, \qquad 0 \le s_i < L, \qquad e_i \le L.$$

- Exactly one layer-count is chosen. The range must lie inside the model's L layers.
- Memory enters as k, the maximum number of layers the GPU can hold. Table 6 has no KV-byte variable.
- If $s_j < e_i < e_j$, traffic from i runs only layers $[e_i, e_j)$ on j. The paper calls this partial inference.
- The solver may be run with partial inference disabled. Then a link is valid only when $e_i = s_j$.

GPU memory limits how many layers are placed. KV capacity is enforced later, at schedule time.

Source: Sec. 4.4, Constraint 1, and the partial-inference remark.

Flow and Compute Constraints

HELIX / MILP

Constraint 2: conservation at node i.

$$\sum_{u} f_{u,i} = \sum_{v} f_{i,v}.$$

Constraint 3: profiled inference throughput.

$$\sum_{u} f_{u,i} \leq \sum_{j=1}^{k} b_{ij}\, T_j.$$

- T_j is a profiled constant: tokens per second when this node holds j layers.
- The binaries b_{ij} pick exactly one of those profiled rates.
- Flow is still also limited by the network edges on the next slide.

A node cannot be assigned more token flow than the profiled rate for the layer count it actually holds.

Source: Sec. 4.4, Constraints 2–3. Sums skip node i itself, as stated in the paper.

Connectivity and Network Constraints

HELIX / MILP

Constraint 4, compacted. A coordinator link into i exists only if i holds the first layer:

$$s_i \le L\,(1 - d_{\text{source},i}). \qquad L\,d_{i,\text{sink}} \le e_i.$$

Between compute nodes, $d_{ij} = 1$ only if $s_j \le e_i < e_j$. Two auxiliary binaries linearize that test, and $d_{ij} \le 0.5\,cond^1 + 0.5\,cond^2$.

Constraint 5: transmission throughput.

$$f_{ij} \le d_{ij}\,S_{ij}.$$

S_{ij} is the profiled token rate of that link, from Sec. 4.3. Invalid links carry zero flow.

Routing variables cannot use a hop that the layer ranges do not justify.

Source: Sec. 4.4, Constraints 4–5. The two-auxiliary form is the paper's linearization.

Converting Max-Flow Weights into Scheduling

HELIX / RUNTIME

IWRR

Interleaved weighted round-robin

Each vertex has a scheduler. A neighbor's weight is the max-flow value on that outgoing edge.

THE WALK

Coordinator, then each node

Pick stage 1 from the coordinator. Pick stage 2 from that node's scheduler. Stop when the path is complete.

WHY INTERLEAVE

Avoid bursts

Frequency follows the weight, but choices are interleaved so one heavy edge is not drained in a burst.

MASK

Before the walk

Nodes over the KV high-water mark are removed from the candidate list. This avoids scheduling onto a full KV cache.

The MILP decides the rates. IWRR is the online rule that realizes those rates.

Source: Sec. 5.1. IWRR follows Tabatabaee et al. as cited by the paper.

Pipeline Scheduling Example

HELIX / RUNTIME

- Vertices are compute nodes. An edge exists only if the MILP placement made it valid.
- The number on an edge is flow in the max-flow solution, not link capacity.
- The figure's two example pipelines are $1 \rightarrow 5 \rightarrow 9$ and $2 \rightarrow 6 \rightarrow 8$.
- Those paths can share the cluster. They are not two frozen replicas.

Read the integers as scheduled token rates, then read the paths as two requests.

Source: Sec. 5.1, Fig. 4. PDF p. 8.

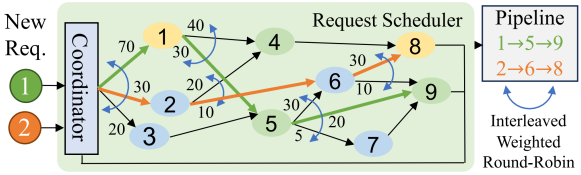


Figure 4. Topology graph of a cluster, where each vertex is a compute node, and each edge is a valid network connection. Numbers over edges represent the flow over the network connection in the max flow solution. The pipelines used to schedule requests 1 and 2 are shown on the right.

High GPU-Heterogeneity Results

HELIX / RESULTS

- Simulator, LLaMA 70B, 42 nodes, 7 types. V100, T4, and 2×T4 cannot form a pipeline alone.
- SP drops those machines. SP+ adds a mixed pipeline that uses them.
- Vs. Swarm / SP / SP+, offline throughput is 1.37×, 2.91×, 2.24×.
- Online: 1.48×, 3.29×, 2.54×. Prompt latency changes by −17%, −1%, −7%.
- Helix's decode latency is slightly higher: baselines leave slow GPUs idle.

The abstract's 3.3× is the 3.29× online gain versus SP, rounded.

Source: Sec. 6.5, Fig. 8. PDF p. 10. Not a prototype measurement.

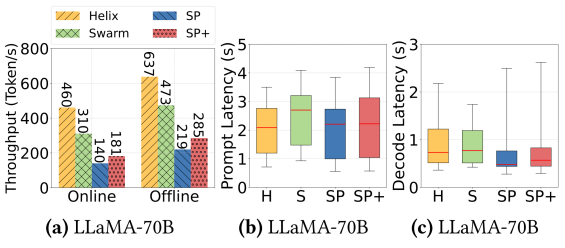
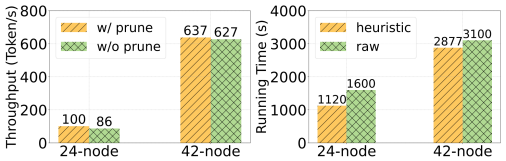


Figure 8. High GPU-heterogeneity clusters setup: (a) decode throughput, **(b - c)** prompt and decode latency for online serving. "H" stands for Helix, "S" stands for Swarm.

MILP Optimization Ablation

HELIX / ABLATION

- Offline LLaMA 70B. 24-node is the geo setup; 42-node is the high-heterogeneity setup.
- Degree-12 pruning drops 50% and 72% of the links, and 36% and 46% of the problem size.
- With pruning, decode throughput is 16% and 2% higher. The paper attributes this to an easier search under a time limit.
- Warm start finds the same placement, in 43% and 8% less wall time.



(a) Cluster pruning. **(b)** Initial values.

Figure 11. Ablation study on MILP optimization.

	With pruning	Without
24-node	876 var, 1122 cstr	1376 var, 1848 cstr
42-node	2144 var, 2772 cstr	4004 var, 5502 cstr

Pruning changes the search problem. It is not a proof that the deleted links were useless.

Source: Sec. 6.8, Fig. 11, Table 8. PDF p. 12. Speedups vary by instance, as the paper notes.

Solver Time and Solution Quality

HELIX / ABLATION

- Instance: LLaMA 30B on 4 L4s and 6 T4s. This is smaller than the 24- and 42-node runs.
- The best feasible placement reaches the optimum in under 5 minutes.
- Certifying that optimum takes more than an hour of branch-and-bound.
- The red dotted line is that cluster's optimal throughput. The upper bound tightens as cuts are added.

Early stop is reasonable here. Fig. 12 does not certify the 42-node plan.

Source: Sec. 6.9, Fig. 12. PDF p. 13.

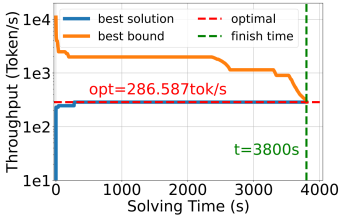


Figure 12. Best solution and best upper bound found by the MILP solver relative to solving time. The red dotted line marks the optimal throughput for this cluster.

PTX Instrumentation and Address Ranges

TESSERA / ANALYZER

1. Add an instrumentation buffer to the kernel parameters and load it with `ld.param`.
2. Around each `ld.global` / `st.global`, emit `atom.global.min` and `atom.global.max` so threads agree on one range.
3. After the launch, align each instruction's [min, max] interval with buffer base addresses. The opcode says read or write.
4. Recompile the patched PTX with `ptxas`. The probe engine is built on NEUTRINO.

If a kernel touches a global that is not a launch argument, the base address is unknown. Tessera then refuses to disaggregate that kernel. The paper cites a study that this pattern is absent in PyTorch and vLLM, and present in one Rodinia kernel.

The analyzer records touched address ranges. It does not reconstruct a kernel's arithmetic.

Source: Sec. III-A and Sec. IV. Listing 1 is the paper's instrumentation sketch.

Extracting RAW Dependencies

TESSERA / ANALYZER

- Execution order is preserved, so only Read-After-Write edges can change the result. WAR and WAW are not the edges Tessera schedules.
- A registry stores the last writer of each buffer. A later reader adds an edge from that writer, annotated with the measured byte size.
- Asynchronous copies are nodes too. Their source, destination, and size come from the API.
- One CUDA graph gets one dependency graph. Dynamic shapes become several graphs, not one graph with a runtime branch.
- Frameworks without CUDA Graph can record a trace and replay it under stream capture. That tool is preprocessing, not the online scheduler.

The graph is a static RAW graph per captured execution, plus the byte size of each edge.

Source: Sec. III-A. “DDG” is the paper’s term for this graph.

Cross-Iteration Dependencies and KV State

TESSERA / ANALYZER

- The default graph is intra-iteration. KV cache is written in one iteration and read in a later one.
- The analyzer flags those buffers by profiling more than one iteration.
- At runtime each GPU keeps a replica. Only the delta is copied, asynchronously.
- Two NCCL communicators separate this background copy from the latency-critical cut-edge transfers. Dependency traffic gets the higher class.
- The paper says delta replication is under 0.1% of latency, citing prior prefill/decode work rather than a new Tessera measurement.

KV state is replicated and updated in the background. It is not a cut edge on the per-iteration critical path.

Source: Sec. III-A and Sec. IV. The 0.1% figure is attributed to prior work [9], [10].

Offline Throughput-Oriented MILP

TESSERA / PLANNER

$$\sum_{g \in G} x_{k,g} = 1, \qquad T_g = \sum_{k \in K} t_{k,g}\, x_{k,g}, \qquad W_g = \max(T_g, M_g), \qquad \min \max_{g \in G} W_g.$$

- T_g is compute time on GPU g. M_g, on the next slide, is communication time into g.
- W_g is the slower of the two. Steady-state throughput is then one request per $\max_g W_g$.
- This is the offline objective. It is the wrong objective when too few requests are in flight to hide transfers.
- Gurobi solves it offline. A reported case, GPT-oss 20B on A100+L40s, has about 500 kernels and finishes one graph in about 20 ms.

The MILP balances GPUs by minimizing the slowest stage. It does not maximize a sum of kernel speeds.

Source: Sec. III-B. The 20 ms figure is for $|G| = 2$, $|K| \approx 500$; all patterns take about 5 s.

Communication and Load-Balancing Constraints

TESSERA / PLANNER

$$c_{ij}^{u,g} = \ell_{u,g} + \frac{d_{ij}}{bw_{u,g}}, \qquad M_g = \sum_{(i,j)\in E} \sum_{u\in G\setminus\{g\}} c_{ij}^{u,g}\, y_{ij}^{u,g}.$$

- $y_{ij}^{u,g} = x_{i,u}x_{j,g}$ is 1 when the producer is on u and the consumer is on g. The product is linearized with the standard Boolean replacement.
- Load balance is not a separate variance constraint. Minimizing $\max_g W_g$ already penalizes the busiest GPU.
- Per-GPU memory capacity is absent. At initialization every GPU receives a full copy of the weights.
- Offline, once the policy is fixed, unused weight buffers can be reclaimed. The paper does not report that reclamation in the evaluation tables.

Communication is bytes over profiled bandwidth, plus a base latency. Memory is a runtime caveat, not a constraint.

Source: Sec. III-B and Sec. IV. Linearization cites Nemhauser and Wolsey.

Online Latency-Oriented Scheduling

TESSERA / PLANNER

Under low load the pipeline cannot hide transfers. The objective adds kernel time and cut-edge transfer time:

$$\min \sum_{k,g} t_{k,g} x_{k,g} + \sum_{(i,j)} \sum_{u} \sum_{g \neq u} c_{ij}^{u,g} \, y_{ij}^{u,g}.$$

- There is no $\max(T, M)$ here. Overlap is not assumed.
- The same Gurobi solve is offline. Workers only look up the stored policy.
- If communication is expensive, this objective keeps dependent kernels on the same GPU. Fig. 11 shows that behavior at 25 Gb/s.

Latency mode shortens one request's critical path. It is not the throughput MILP with a different label.

Source: Sec. III-B. The sum is the paper's low-load decomposition, compacted for the slide.

GPU Worker Execution

TESSERA / EXECUTION

- Fig. 5 is GPU B from the six-kernel example. Compute and communication use different streams.
- R_{ij} receives kernel i's output for kernel j. S_{ij} sends it. CUDA events order the two streams.
- Receives are posted before the consumer kernel. Sends are posted after the producer if the edge is cut.
- Sends and receives are GPU-side kernels via NCCL's GPUDirect Async (IBGDA), so the CPU is not on the critical path.

A worker runs its own kernels and the cut-edge messages. It does not run the whole model.

Source: Sec. III-C, Fig. 5. PDF p. 6.

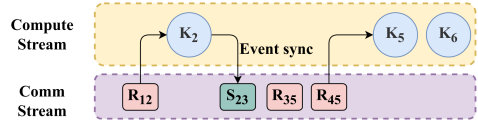


Fig. 5: An execution example of the GPU worker. Compute kernels and communication operations run on separate streams, synchronized via CUDA events. R_{ij} denotes receiving the output of kernel i for kernel j, S_{ij} denotes sending data from kernel i to kernel j.

Online Monitor Signals

TESSERA / RUNTIME

- The monitor records per-request latency, communication time, and per-kernel-group latency.
- A kernel group is the run of kernels between two consecutive communication operations. Individual kernels are not timed online.
- $\bar{L}_{\mathrm{req}}$ is average end-to-end request latency. $\bar{L}_{\mathrm{exec}}$ sums compute and communication and excludes queueing.
- The pressure signal is the ratio $\bar{L}_{\mathrm{req}}/\bar{L}_{\mathrm{exec}}$.
- A low ratio means little queueing, so the latency policy is preferred. A high ratio means queueing dominates, so the throughput policy is preferred.

The switch looks at queueing pressure, not at a hand-named GPU counter.

Source: Sec. III-D. Kernel group is the paper's definition.

Policy Switching and Sensitivity Parameters

TESSERA / RUNTIME

WINDOW W

How often to decide

The ratio is computed at window boundaries. The chosen defaults are $W = 300$ ms and $\beta = 1.5$.

SWITCH COST

About 30 ms

Workers synchronize on an iteration boundary. The 30 ms is the paper's reported stall, not a bound.

THRESHOLD β

When queueing wins

Crossing β selects the throughput policy. Staying below it selects the latency policy.

WHERE IT WAS TUNED

One trace

GPT-oss 20B, 360 s of the Azure Conversation trace. Fig. 10 is the sensitivity plot.

W and β are the only two knobs the paper names. Aggressive and lazy settings both increase latency.

Source: Sec. III-D and Sec. V-D. Sensitivity measurements are in Fig. 10 of the paper.

Network Bandwidth and MILP Analysis

TESSERA / DRILL-DOWN

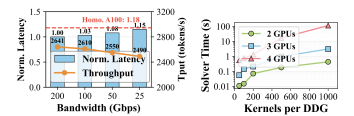


Fig. 11: (a) Impact of network bandwidth on GPT-oss 20B with A100+L40s. (b) MILP solver time.

Fig. 11. (a) GPT-oss 20B, A100+L40s. (b) Gurobi time vs. kernels per graph.

(a) Throughput at 200 / 100 / 50 / 25 Gb/s: 2641 / 2610 / 2550 / 2490 tokens/s. The 25 Gb/s drop is under 6%. Normalized latency: 1.00 / 1.03 / 1.08 / 1.15, still below the homogeneous A100 line at 1.18.

(b) Text, 2 GPUs: 0.07 s at 200 kernels and 0.43 s at 1000 kernels. A DeepSeek-V3 671B forward graph of about 1500 kernels solves within 1 s. That is a solver stress test, not a serving result. Time grows with the number of GPU types; repeated layers are solved once.

At 25 Gb/s the offline pipeline still hides most of the extra delay. The latency policy moves work off the slow cut.

Source: Sec. V-D, Fig. 11. PDF p. 10. Do not read panel (b) as an end-to-end DeepSeek result.

Cost Efficiency of Heterogeneous Pairing

TESSERA / RESULTS

Table III. Perf/$: mean throughput over the five workloads, divided by rental cost, then normalized to the left GPU alone. A pair pays for both GPUs.

System	A100+L40s	H100+RTX Pro 6000	B200+H100
Homogeneous left	1.00	1.00	1.00
Homogeneous right	1.18	2.01	0.78
PD disaggregation	0.87	1.00	0.70
AF disaggregation	1.02	1.07	0.74
Tessera	1.21	1.64	1.01

- Vs. PD: 1.5× cost efficiency on average, up to 1.6×. Vs. AF: 1.4× on average, up to 1.5×.
- On H100+RTX Pro 6000, Tessera is 1.64× the Perf/$ of H100 alone and 2.3× its throughput, above two H100s, at price $2.9 + 1.2$ versus 2×2.9.

The pair can beat a single expensive GPU on Perf/$. The prices are Table I's relative rentals, not a new quote.

Source: Sec. V-B, Table III. The 2.3× sentence is the cost-efficiency paragraph, not the PD maximum.

Generality Across Model Families

TESSERA / RESULTS

What was run	Models	What it does not show
Offline pairs	Llama-3 8B, GPT-oss 20B, Qwen2.5-VL 7B, Mamba-Codestral 7B, SD 3.5	PD or AF is inapplicable on some families
Online SLO	GPT-oss 20B only	Latency for SSM, MLLM, or diffusion
Cluster, offline	GPT-oss 20B and Qwen-3 235B	The single-pair Fig. 6 suite
Solver stress test	DeepSeek-V3 671B, one forward graph	End-to-end serving

- AF needs an attention block, so Mamba and diffusion are out. PD needs a prefill/decode split, so diffusion is out.
- The motivation profiles say "Mamba 7B"; the evaluated SSM is Mamba-Codestral 7B. Do not merge the two names.

"Four families" means the offline pair study. Online evidence is one LLM.

Source: Sec. II-B, Sec. V-A–V-D. DeepSeek-V3 is the solver paragraph in Sec. V-D.